

Lesson 01 · The .NET Platform Map

C# · .NET · VB for a Kotlin/Java, TypeScript & Python architect — runtime, languages, versions and toolchain

Contents

1 · Why this track exists — and who it is written for	1
1.1 · The numbers to leave this lesson with	1
2 · The platform in one picture	2
2.1 · From source file to running process	2
2.2 · Runtime pieces — the JVM parts under their .NET names	3
2.3 · Concept map — JVM and friends → .NET	3
3 · Versions — how to read a .NET codebase's age	4
3.1 · Support windows of modern .NET	4
3.2 · Support length by release	4
3.3 · Where this track's study time goes	4
3.4 · Target framework monikers you will find in project files	5
3.5 · What am I looking at? — dating a legacy repository	5
4 · Three languages, one runtime	5
4.5 · Language feature matrix — C#, VB and the languages you know	7
5 · The toolchain — one CLI, five ways to ship	7
5.1 · The dotnet CLI next to the tools you use today	7
5.3 · Five ways to ship the same code	9
5.4 · What happens on dotnet run (simplified)	9
6 · Your transfer map and learning path	9
6.1 · Transfer map — how much of each lesson you already know	10
6.2 · Learning path	10
6.3 · The twelve lessons	11
7 · Hands-on — build and run the lesson-01 samples	11
7.4 · Lines of code in the lesson-01 samples	13

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why this track exists — and who it is written for

This track is written for one reader: an architect with eight years across Kotlin/Java, TypeScript, Python and Dart who has never had to own a .NET codebase — and keeps meeting one. Insurance and banking estates run a large share of their line-of-business systems on .NET, and the Windows (IIS) workloads that sit beside ECS Fargate and Lambda in a migration inventory are where .NET Framework most often hides. Reading that code, sizing its migration and designing its replacement is architecture work; this track gives you the vocabulary to do it first-hand.

The approach is map, don't restart. Every lesson starts from something you have already built — a Kotlin mono-repo of 50–120+ modules, a Spring WebFlux service, a Python Lambda tuned to ~175 ms cold starts, a 100%-coverage TDD suite, an Entra ID single sign-on integration — and shows the .NET shape of the same idea. Chips say whether an idea transfers as-is (• Same idea), only changes its name (• Renamed), genuinely behaves differently (• Different) or looks the same and bites (• Trap).

Why C# and VB together. New .NET work is written in C#. Estates that have been running since the 2000s also carry Visual Basic .NET, and a modernisation architect has to read it, keep it running and plan its exit. VB appears beside C# wherever the difference teaches something, and lesson 06 (VB.NET for Legacy Estates) is dedicated to it.

Every code panel is real. C# and VB snippets are cut from sample projects in this repository by #region name, and 0-script/verify_samples.py builds, tests and runs those projects on the .NET 10 SDK. A snippet that did not compile could not appear here.

1.1 · The numbers to leave this lesson with

.NET 10 · LTS to Nov 2028 supported until 14 Nov 2028 · verified	.NET 8 & 9 SUPPORT 57 days end 10 Nov 2026 · counted from 2026-09-14	LANGUAGES, ONE RUNTIME 3 C# · Visual Basic · F#	THIS TRACK 71 h 12 lessons · study-time estimate	LESSON 01 SAMPLES 55 lines C# + VB · measured at build
---	---	---	---	--

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# VB Java/Kotlin runtime tooling legacy

i How to read every lesson in this track

- Each numbered section **leads with its point**; evidence, code and diagrams follow. Reading only the first paragraph of each section gives you the whole argument.

2. Facts that change with releases — support dates, runtime versions, language features — are marked **• Verified** and link to the official source. Judgements such as study hours are marked **• Estimate** ; numbers computed from this repository at build time are marked **• Measured** .
3. Code panels are labelled with the file and #region they were cut from, so you can open the sample and run it. Java, Kotlin, TypeScript and Python panels are for comparison only and are not compiled.
4. Every lesson ends with **Check yourself** questions. If you can answer them from memory, move on.

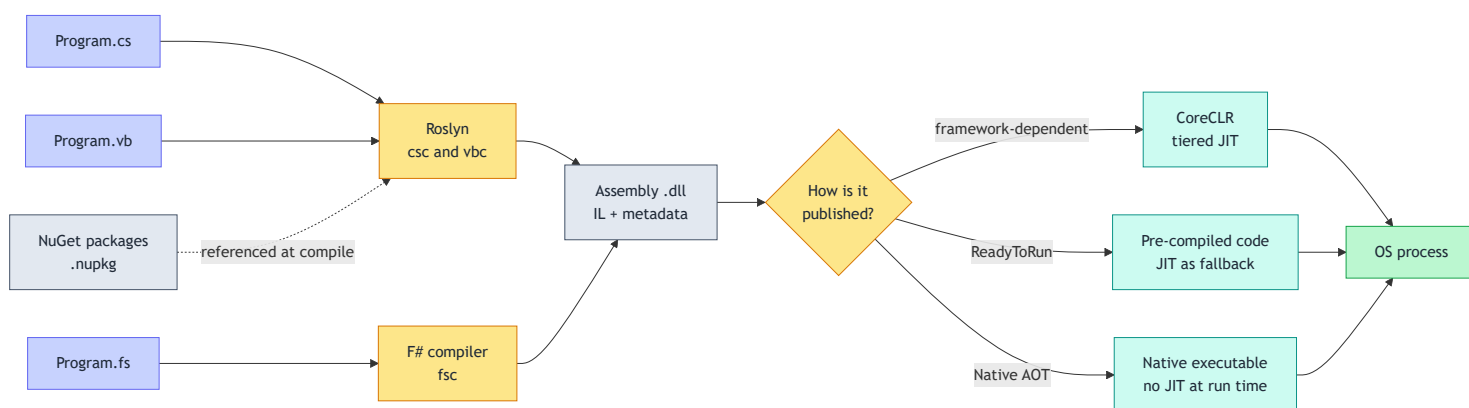
2 · The platform in one picture

.NET is a runtime plus a class library plus compilers — the same three-part shape as the JDK, with one decisive difference: the runtime was designed for many languages from day one. C#, Visual Basic and F# compile to the same intermediate language (IL), load into the same runtime and share the same type system. A VB program can call a C# library with no bridge, adapter or marshalling layer — the lesson-01 samples do exactly that.

The pieces map almost one-to-one onto the JVM. CoreCLR is HotSpot. IL inside an assembly is bytecode inside a JAR. The Base Class Library is the JDK class library. NuGet is Maven Central. Roslyn — the C# and VB compiler, itself written in C# — is javac with a public API that analyzers and IDEs call directly.

Where it stops being the JVM. Generics are *reified*: `List<int>` is a real type at run time and stores unboxed integers. User-defined value types (struct) live inline, not on the heap. Exceptions are all unchecked. And the compiled output does not have to stay IL: ReadyToRun pre-compiles it and Native AOT removes the JIT altogether — GraalVM native-image is the nearest thing you know.

2.1 · From source file to running process



indigo = source · amber = compiler or decision · slate = artefact · teal = runtime path · green = the OS process

1 CoreCLR — the Common Language Runtime runtime • Renamed

Loads assemblies, verifies and JIT-compiles IL, runs the garbage collector, owns threads and exceptions.

JVM	HotSpot / OpenJ9
DIFFERS	One runtime serves every .NET language — C#, VB and F# share types directly
VARIANTS	CoreCLR on servers and desktop; a Mono-derived runtime on Android, iOS and WebAssembly
OLD NAME	The .NET Framework CLR is a separate, Windows-only runtime — see §3

2 IL, metadata and assemblies runtime • Renamed

The compiler emits intermediate language plus rich type metadata into an assembly.

JVM	Bytecode in .class files, packaged into a JAR
.NET	IL + metadata in one assembly (.dll) that holds many types
INSPECT	ILSpy or ildasm — the javap of .NET
TRAP	The .exe beside your .dll is an <i>apphost</i> launcher; the program is still IL

3 Tiered JIT, dynamic PGO and ReadyToRun runtime • Same idea

Start fast with a quick JIT, then recompile hot methods with full optimisation.

JVM	C1 → C2 tiered compilation
.NET	Tier-0 quick JIT → tier-1 optimised JIT; on by default since .NET Core 3.0
PGO	Dynamic profile-guided optimisation is on by default since .NET 8
R2R	ReadyToRun ships pre-compiled native code inside the assembly to cut start-up; JIT still available

4 Garbage collector and value types runtime • Different

A generational, compacting GC — plus a type system that avoids many allocations in the first place.

JVM	G1 / ZGC / Parallel collectors
MODES	Workstation or Server GC; ASP.NET Core apps default to Server GC
STRUCTS	Value types live inline (stack or inside an object) and copy on assignment
SAMPLE	The lesson-01 console app reports <code>ServerGc = False</code> — a console default

5 Base Class Library and NuGet tooling • Renamed

A large standard library, and one public package registry.

JDK	java.base, java.net.http, java.time ...
.NET	System.*, System.Net.Http, System.Text.Json, System.Collections.Generic
REGISTRY	nuget.org ≈ Maven Central; dependencies are PackageReference items in the project file
LOCK FILE	packages.lock.json is opt-in (RestorePackagesWithLockFile) — unlike npm

2.3 · Concept map — JVM and friends → .NET

You already know	In .NET	Kind	Watch for
JVM (HotSpot)	CoreCLR	• Renamed	Multi-language by design — not a VM for one language
Bytecode in .class	IL in an assembly (.dll)	• Renamed	An assembly is closer to a JAR than to a .class
JAR / fat JAR	Assembly / self-contained publish	• Renamed	The .exe apphost is a launcher, not a native build
JDK class library	Base Class Library (BCL)	• Same idea	Namespaces start with System.
Maven Central / npm	NuGet (nuget.org)	• Renamed	No separate pom or package.json — it is all in the project file
pom.xml / build.gradle.kts	.csproj / .vbproj (MSBuild)	• Different	Declarative XML; shared logic goes in Directory.Build.props — lesson 07 (Solutions, MSBuild & Mono-repos)
Generics with type erasure	Reified generics	• Trap	<code>typeof(T)</code> works; <code>List<int></code> never boxes
Primitive wrappers, autoboxing	Value types (struct)	• Different	Mutating a copy of a struct changes nothing — a classic bug
Checked exceptions	Unchecked exceptions only	• Different	No throws; document with <code>/// <exception></code>
Kotlin data class / Java record	C# record	• Same idea	<code>with = copy()</code> ; VB can use records but not declare them

You already know	In .NET	Kind	Watch for
Kotlin coroutines / WebFlux	async/await + Task	• Different	lesson 05 (Async, Tasks & Concurrency)
GraalVM native-image	Native AOT	• Same idea	Same closed-world trade-off: reflection needs annotations
SDKMAN / .nvrc / pyenv	global.json	• Renamed	Pins the SDK, not the runtime; rollForward decides
Spring Boot	ASP.NET Core	• Different	The DI container is built in — lesson 08 (ASP.NET Core Web APIs)
Java EE app on WebSphere	ASP.NET on IIS (.NET Framework)	• Same idea	System.Web does not exist on modern .NET — port, don't lift
JUnit + Mockito	xUnit / NUnit / MSTest + NSubstitute / Moq	• Renamed	lesson 10 (Testing, TDD & 100% Coverage)

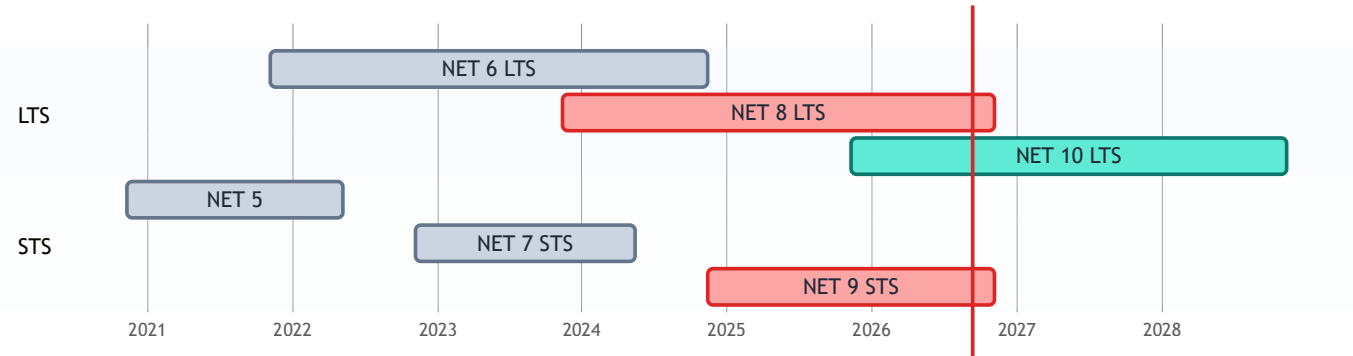
3 · Versions — how to read a .NET codebase's age

There are two .NETs, and a codebase's project file tells you which one it needs in the first ten lines. .NET Framework (1.0 in 2002 → 4.8.1 in 2022) is Windows-only, ships as a Windows component and receives no new major versions. .NET — born as .NET Core 1.0 in 2016 and renamed at .NET 5 in 2020, skipping “4” to avoid confusion — is cross-platform, open source and ships every November.

The release rhythm is predictable. Even-numbered releases are Long Term Support (36 months); odd-numbered releases are Standard Term Support, extended from 18 to 24 months starting with .NET 9. That extension makes .NET 8 (LTS) and .NET 9 (STS) leave support on the same day, 10 November 2026. .NET 10 shipped on 11 November 2025 and is supported until 14 November 2028 ([.NET releases and support](#) · [.NET 8 and .NET 9 end of support](#)).

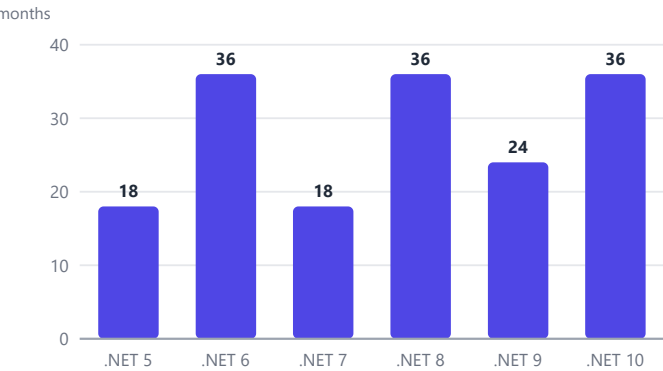
What that means for an architect in September 2026. New work targets net10.0. Anything on net8.0 or net9.0 has a hard upgrade deadline this quarter. Anything on .NET Core 3.1, .NET 5, 6 or 7 is already unsupported. And a .NET Framework 4.8.1 application is still supported — for as long as the Windows version it runs on is — but it will never get a new runtime feature, which is a migration argument rather than a security one.

3.1 · Support windows of modern .NET



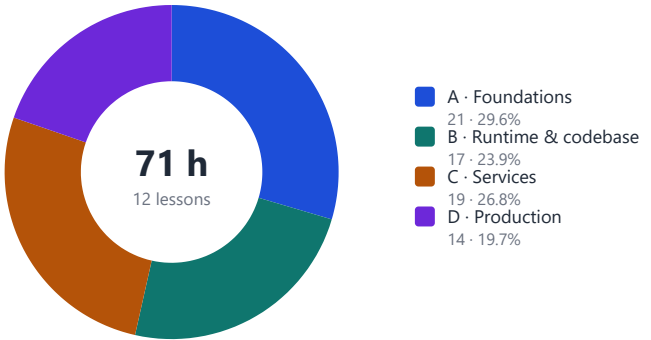
grey = out of support · red = ends 10 Nov 2026 · teal = current LTS · red vertical line = the day this PDF was built

3.2 · Support length by release



months from GA to end of support · verified against Microsoft's lifecycle pages

3.3 · Where this track's study time goes



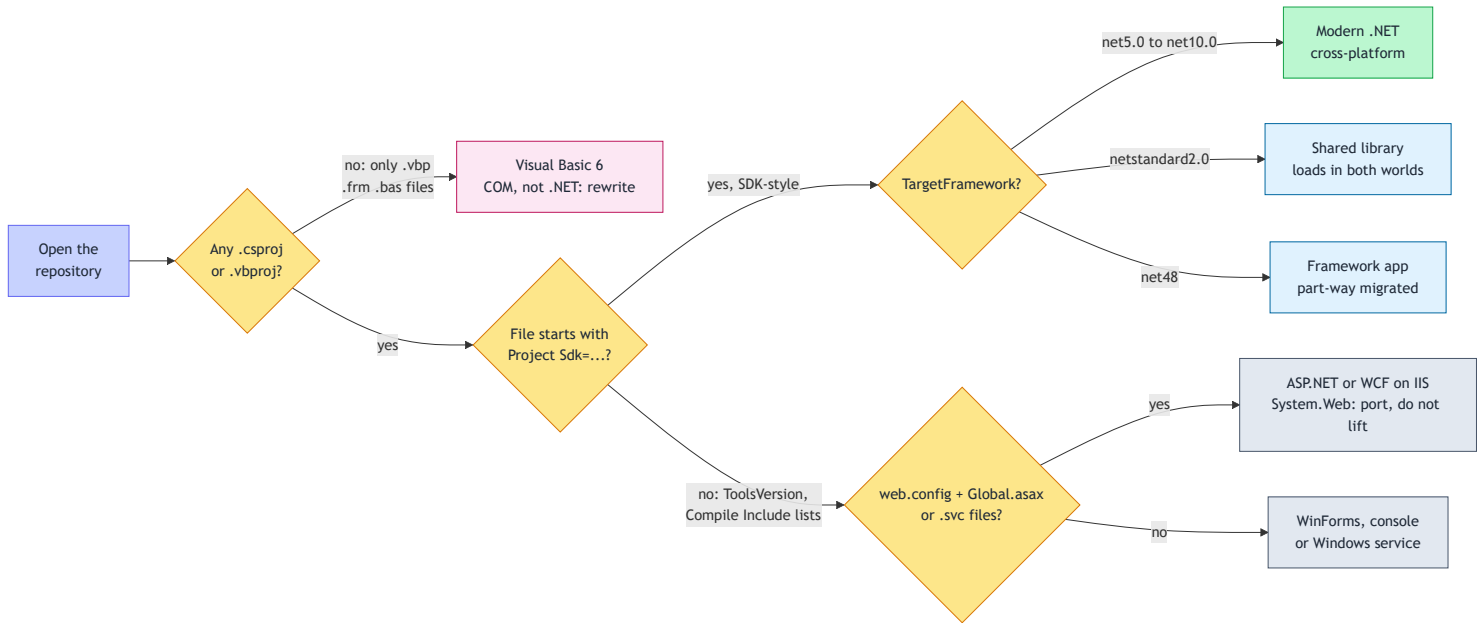
suggested hours per learning-path phase · estimate

Foundations take the largest share on purpose. Language and type-system habits are what make C# read like C# rather than Java written in C# syntax.

3.4 · Target framework monikers you will find in project files

TFM	What it targets	Where you meet it	Status
net48 · net481	.NET Framework 4.8 / 4.8.1 — Windows only	ASP.NET, WCF and WinForms apps; IIS on Windows servers	• Legacy · follows Windows
netstandard2.0	An API contract, not a runtime	Shared libraries that must load in both .NET Framework and modern .NET	• Bridge
netcoreapp3.1	.NET Core 3.1	First-wave migrations	• Ended Dec 2022
net6.0 · net7.0	.NET 6 LTS · .NET 7 STS	Services built 2021–2023	• Ended 2024
net8.0 · net9.0	.NET 8 LTS · .NET 9 STS	Most services running today	• Ends 10 Nov 2026
net10.0	.NET 10 LTS	New work — every sample in this track	• Supported to Nov 2028
net10.0-windows	.NET 10 plus Windows-only APIs	Modernised WinForms / WPF desktop apps	• Windows only

3.5 · What am I looking at? — dating a legacy repository



indigo = start · amber = question · green = modern · sky = bridge · slate = .NET Framework · rose = not .NET at all

⚠ The 10 November 2026 double cut-off

- 1. **.NET 8 (LTS) and .NET 9 (STS) stop receiving security fixes on the same day.** Upgrading from 8 to 9 buys nothing; the only supported destination is .NET 10.
- 2. **Start the inventory with the TFM, not the language.** A VB.NET service on net8.0 is a one-line retarget plus a test run; a C# service on net48 is a port.
- 3. **Check the container base images too.** An app retargeted to net10.0 but still built on an 8.0 runtime image is still running .NET 8.
- 4. Source: [.NET 8 and .NET 9 end of support](#).

4 · Three languages, one runtime

C# is where .NET evolves; Visual Basic is deliberately stable; F# is the functional option. C# 14 ships with .NET 10 and keeps adding syntax — extension members, the `field` keyword, null-conditional assignment ([What's new in C# 14](#)). Microsoft's stated strategy for Visual Basic is *consumption-only*: VB code can use new .NET APIs and types, but the language will generally not add syntax to *define* things that need new language support ([Visual Basic language strategy](#) · [.NET managed languages strategy](#)).

What that means in practice. A VB project can reference a C# library and call it as if it were written in VB — same types, same exceptions, same debugger. It can read a C# record, compare it and pass it on; it cannot declare a record or use `with` to copy one. The samples below are the proof: one C# class library, consumed unchanged by a C# console app and a VB console app.

What an architect does with this. Mixed-language solutions are normal and cheap — there is no FFI to design. Migration from VB to C# can therefore be incremental: new code in C# libraries, old VB calling it, one project converted at a time. [lesson 06 \(VB.NET for Legacy Estates\)](#) turns this into a plan.

4.1 · The shared C# library — a positional record

C#samples/L01.PlatformKit/PlatformReport.cs · #region report-record

```
/// <summary>An immutable snapshot of the runtime a program is executing on.</summary>
public sealed record PlatformReport(
    string Framework,           // ".NET 10.0.x" – the product you installed
    string RuntimeVersion,      // CLR version, e.g. 10.0.0
    string OperatingSystem,
    Architecture ProcessArchitecture,
    bool ServerGc,              // workstation vs server garbage collector
    bool DynamicCodeCompiled,   // true = JIT is on; false under Native AOT
    string MetadataVersion,     // "v4.0.30319" on every modern .NET (a trap!)
    string CallerAssembly)
{
    public static PlatformReport Capture(Assembly caller) => new(
        Framework: RuntimeInformation.FrameworkDescription,
        RuntimeVersion: Environment.Version.ToString(),
        OperatingSystem: RuntimeInformation.OSDescription,
        ProcessArchitecture: RuntimeInformation.ProcessArchitecture,
        ServerGc: GCSettings.IsServerGC,
        DynamicCodeCompiled: RuntimeFeature.IsDynamicCodeCompiled,
        MetadataVersion: caller.ImageRuntimeVersion,
        CallerAssembly: caller.GetName().Name ?? "(unnamed)");
}
```

One declaration gives you a constructor, read-only properties, value equality, ToString() and deconstruction — Kotlin's data class in C#. Note MetadataVersion: every modern .NET assembly reports v4.0.30319, the metadata format version, not the .NET version.

4.2 · The same program in C# and in Visual Basic

C#samples/L01.CSharpApp/Program.cs · #region program

```
var r = PlatformReport.Capture(
    Assembly.GetExecutingAssembly());

Console.WriteLine($"Hello from C# on {r.Framework}");
Console.WriteLine($"  os      {r.OperatingSystem}");
Console.WriteLine($"  arch    {r.ProcessArchitecture}");
Console.WriteLine($"  JIT on   {r.DynamicCodeCompiled}");
Console.WriteLine($"  metadata {r.MetadataVersion}");
Console.WriteLine($"  assembly {r.CallerAssembly}");
```

Visual Basicsamples/L01.VbApp/Program.vb · #region program

```
Dim r = PlatformReport.Capture(
    Assembly.GetExecutingAssembly())

Console.WriteLine($"Hello from VB on {r.Framework}")
Console.WriteLine($"  os      {r.OperatingSystem}")
Console.WriteLine($"  arch    {r.ProcessArchitecture}")
Console.WriteLine($"  JIT on   {r.DynamicCodeCompiled}")
Console.WriteLine($"  metadata {r.MetadataVersion}")
Console.WriteLine($"  assembly {r.CallerAssembly}")
```

Same library, same types, same output. C# uses top-level statements; VB wraps the code in a Module with Sub Main. VB is case-insensitive and line-oriented — the Capture() call continues on the next line because it ends with an open parenthesis.

4.3 · Value equality and non-destructive copy — Kotlin vs C#

Kotlin — for comparisonthe idea you already know

```
data class PlatformReport(
    val framework: String,
    val serverGc: Boolean,
    // ...
)

val flipped = r.copy(serverGc = !r.serverGc)
println(" same?   ${r == flipped}") // false
println(" copy?   ${r == r.copy()}") // true
```

C#samples/L01.CSharpApp/Program.cs · #region record-equality

```
// records compare by VALUE, like a Kotlin data class
var flipped = r with { ServerGc = !r.ServerGc };
Console.WriteLine($" same?   {r == flipped}"); // False
Console.WriteLine($" copy?   {r == r with { }}"); // True
```

• Same idea idea, different keyword. with { } is copy(). Because records compare by value, r == r with { } is true even though it is a new object.

4.4 · What Visual Basic can do with that C# record

Visual Basicsamples/L01.VbApp/Program.vb · #region record-equality

```
' VB can USE the C# record (read it, compare it) but has
' no `With` syntax to make a modified copy of it.
Dim same = r.Equals(PlatformReport.Capture(
    Assembly.GetExecutingAssembly()))
Console.WriteLine($"equal?   {same}") True
```

Consumption-only in one screen. VB calls Equals and reads every property; there is no VB syntax for a with copy. For a migration plan this is good news: shared models can move to C# first without breaking their VB callers.

4.5 · Language feature matrix — C#, VB and the languages you know

	C# 14	VB	Java 21	Kotlin 2
Records / data classes	yes	some	yes	yes
Non-destructive copy	yes	—	—	yes
Null-safety in types	yes	—	—	yes
Pattern-matching switch	yes	some	yes	some
async / await style	yes	yes	some	yes
Language-integrated query	yes	yes	some	some
Runtime (reified) generics	yes	yes	—	some
User-defined value types	yes	yes	—	some
Properties	yes	yes	—	yes
Extension members	yes	some	—	yes
XML literals	—	yes	—	—

yes = first-class syntax · some = library, subset or consume-only · — = none · a rubric, not a benchmark

VB matches C# on everything that existed before 2015 and stops there. Its gaps are the post-2019 C# features (records, with, nullable reference types, pattern matching). Its one unique row — XML literals — is also why some legacy VB code is hard to port mechanically. Java's "some" async is virtual threads and `CompletableFuture`; Kotlin's "some" generics are inline reified functions.

Carries over unchanged

- OOP, SOLID and dependency-injection thinking
- Unit testing, TDD and coverage discipline
- Microservice, event-driven and cloud architecture
- Build-pipeline and mono-repo design instincts

Learn fresh

- Value types and struct copy semantics
- `async/await` and `Task` as the concurrency model
- LINQ and expression trees
- MSBuild evaluation and `Directory.Build.props`

Unlearn

- Type-erasure workarounds such as `Class<T>` tokens
- Checked exceptions and `throws` clauses
- Hand-written getters and setters — use properties
- "The .exe is native code" — it is an apphost over IL

5 · The toolchain — one CLI, five ways to ship

The dotnet CLI is Gradle, npm and SDKMAN in one executable. It scaffolds from templates, restores packages, builds, tests, runs, packs and publishes. Its SDK version is pinned per repository by `global.json` — this repository pins 10.0.401 and lets later feature bands roll forward. Restore is implicit in build, test and run, so the everyday loop is two commands.

A project file is declarative MSBuild, not a script. An SDK-style `.csproj` or `.vbproj` is often five lines, because the SDK supplies defaults: every source file in the folder is compiled, packages restore from `nuget.org`, output goes to `bin/`. Shared settings live in `Directory.Build.props`, which MSBuild imports for every project beneath it — this repository uses one to put all build output in `.build/artifacts`. lesson 07 (Solutions, MSBuild & Mono-repos) goes deep.

Publishing is a decision, not a default. The same IL can ship framework-dependent, self-contained, as a single file, pre-compiled with `ReadyToRun`, or as a Native AOT executable. The choice trades start-up time and package size against compatibility with reflection-heavy libraries — the same trade you made choosing between a JAR on a JRE and a GraalVM native image, and the one you tuned when you got Python Lambda packages down to 55 KB.

5.1 · The dotnet CLI next to the tools you use today

Task	Maven / Gradle	npm	.NET CLI	Note
Scaffold	gradle init	npm init	dotnet new console -lang VB	dotnet new list shows templates
Restore dependencies	implicit	npm install	dotnet restore	implicit in build, test and run
Compile	gradle build	npm run build	dotnet build -c Release	Debug is the default configuration
Test	gradle test	npm test	dotnet test	works with xUnit, NUnit and MSTest — lesson 10 (Testing, TDD & 100% Coverage)
Run	gradle bootRun	npm start	dotnet run	.NET 10 also runs a single file: dotnet run app.cs
Package an app	gradle bootJar	—	dotnet publish	-r linux-x64 --self-contained -p:PublishAot=true
Package a library	gradle publish	npm publish	dotnet pack · dotnet nuget push	produces a .nupkg
Add a dependency	edit the build file	npm install x	dotnet add package X	writes a PackageReference
Global tools	SDKMAN candidates	npm i -g	dotnet tool install -g	repo-local tools: dotnet-tools.json
Format / lint	Spotless · ktlint	Prettier · ESLint	dotnet format	rules come from .editorconfig

5.2 · Two project files — the language is the file extension

MSBuild / XML	samples/L01.CSharpApp/L01.CSharpApp.csproj	MSBuild / XML	samples/L01.VbApp/L01.VbApp.vbproj
<pre><Project Sdk="Microsoft.NET.Sdk"> <PropertyGroup> <OutputType>Exe</OutputType> </PropertyGroup> <ItemGroup> <ProjectReference Include="..\L01.PlatformKit\L01.PlatformKit.csproj" /> </ItemGroup> </Project></pre>		<pre><Project Sdk="Microsoft.NET.Sdk"> <PropertyGroup> <OutputType>Exe</OutputType> <RootNamespace>L01.VbApp</RootNamespace> </PropertyGroup> <ItemGroup> <ProjectReference Include="..\L01.PlatformKit\L01.PlatformKit.csproj" /> </ItemGroup> </Project></pre>	

The same MSBuild, the same SDK. Neither file lists a source file or a target framework: the SDK compiles every .cs or .vb in the folder, and net10.0 comes from the repository's Directory.Build.props. The VB app references the C# library exactly the way the C# app does.

6 Framework-dependent tooling • Same idea

Ship your IL and dependencies; the machine supplies the .NET runtime.

- LIKE** A thin JAR on an installed JRE · a Python zip on a Lambda runtime
- SIZE** Smallest package — the lesson-01 VB app is a few kilobytes of IL
- PICK WHEN** AWS Lambda managed runtime, containers on the aspnet image, servers you patch

7 Self-contained tooling • Renamed

Ship the runtime with the app, for one runtime identifier such as linux-x64.

- LIKE** A jlink runtime image
- COST** Tens of megabytes more; runtime patches arrive only when you redeploy
- PICK WHEN** Hosts where you cannot install or control the runtime

8 Single-file tooling • Renamed

Bundle the app (and optionally the runtime) into one executable file.

- LIKE** A fat JAR with a native launcher
- NOTE** Still JIT-compiled at start-up unless combined with ReadyToRun
- PICK WHEN** CLI tools and agents you copy onto machines

9 ReadyToRun runtime • Different

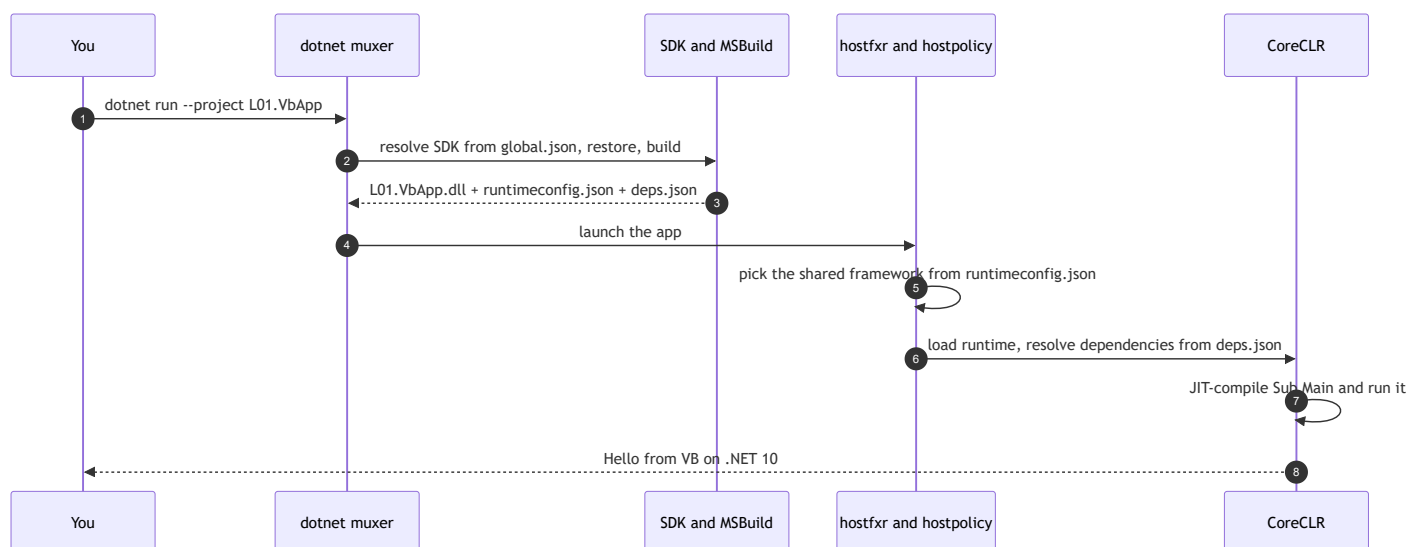
Pre-compile IL to native code at publish time and keep the JIT as a fallback.

- LIKE** An ahead-of-time compilation cache — faster start, same compatibility
- COST** Larger assemblies; tier-1 JIT still re-optimises hot code
- PICK WHEN** Start-up matters but you depend on reflection-heavy libraries

10 Native AOT runtime • Same idea

Compile the whole app to a native executable — no JIT and no IL at run time.

- LIKE** GraalVM native-image — the same closed-world trade-off
- FOR YOU** The .NET answer to the ~175 ms cold starts you tuned in Python Lambdas — lesson 12 (Cloud-Native .NET on AWS & Modernization)
- LIMITS** No run-time code generation; reflection needs trimming annotations
- TELL-TALE** `RuntimeFeature.IsDynamicCodeCompiled` returns false

5.4 · What happens on dotnet run (simplified)

the muxer finds the SDK through `global.json`; the host finds the runtime through `runtimeconfig.json`

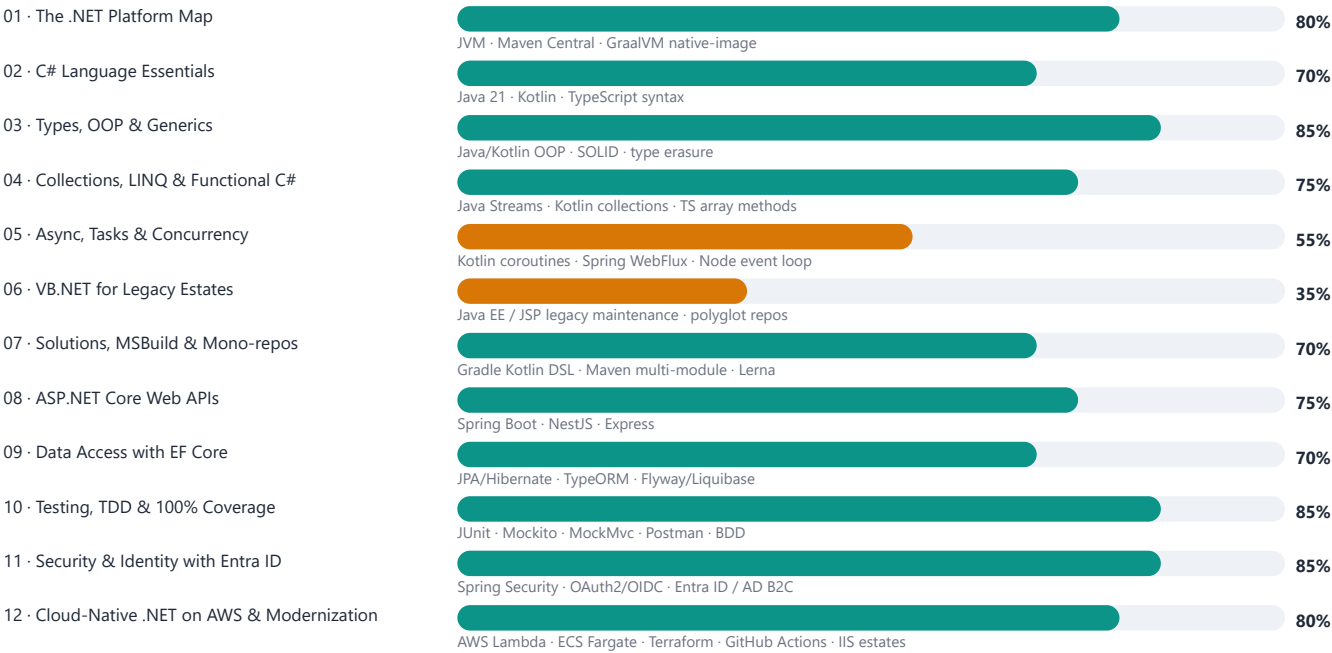
6 · Your transfer map and learning path

Most of this track is translation, and the chart below says where it is not. The estimate for each lesson is the share of its content that a Java/Kotlin, TypeScript and Python engineer already knows under another name. Two lessons fall well below the rest: asynchronous code, where C#'s Task model differs from both coroutines and reactive streams, and VB.NET, where the syntax itself is new.

The path has four phases. *Foundations* makes you fluent in the language. *Runtime & codebase* covers concurrency, legacy VB and how large solutions are built — the three things that decide whether you can read an existing estate. *Services* builds, persists and tests an ASP.NET Core API. *Production* secures it with Entra ID and ships it to AWS, and closes with modernising a Windows / IIS application.

One running example ties the samples together. From lesson 02 onward the samples model a motor-insurance quoting service — vehicles, drivers, quotes, premiums and policies — so each lesson adds a layer to something you already understand rather than starting a new toy.

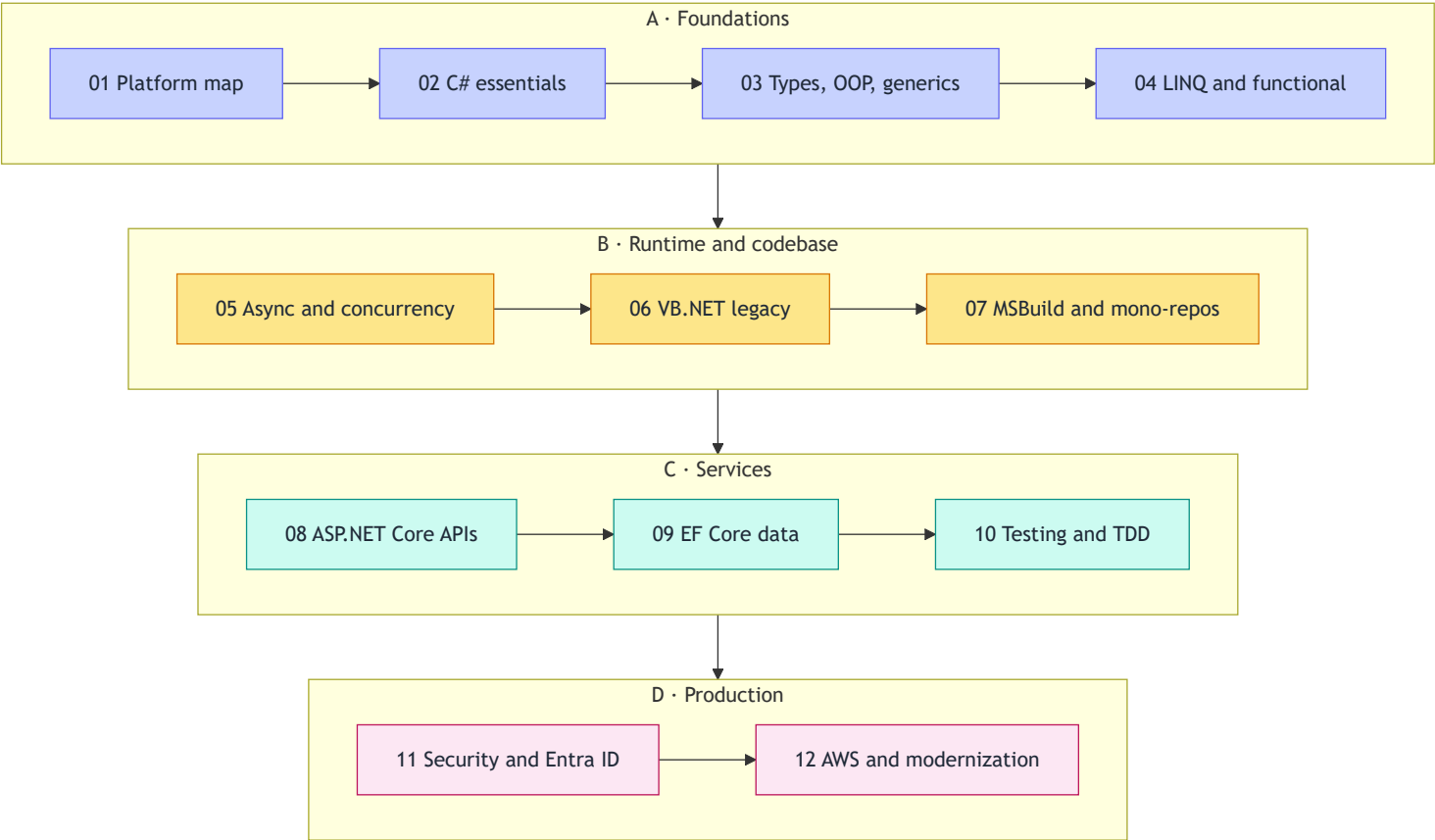
6.1 · Transfer map — how much of each lesson you already know



estimate · red below 34% · amber below 67% · the grey line under each bar names what it maps from

Spend the saved time on lessons 05 and 06. Everything above 70% is mostly vocabulary; async (55%) and VB.NET (35%) are where a senior engineer from another stack most often writes code that compiles and is still wrong.

6.2 · Learning path



arrows = recommended order · colours = the four phases

6.3 · The twelve lessons

#	Lesson	Maps from	Phase	Hours	Transfer
01	The .NET Platform Map	JVM · Maven Central · GraalVM native-image	A · Foundations	4	80%
02	C# Language Essentials	Java 21 · Kotlin · TypeScript syntax	A · Foundations	6	70%
03	Types, OOP & Generics	Java/Kotlin OOP · SOLID · type erasure	A · Foundations	6	85%
04	Collections, LINQ & Functional C#	Java Streams · Kotlin collections · TS array methods	A · Foundations	5	75%
05	Async, Tasks & Concurrency	Kotlin coroutines · Spring WebFlux · Node event loop	B · Runtime & codebase	7	55%
06	VB.NET for Legacy Estates	Java EE / JSP legacy maintenance · polyglot repos	B · Runtime & codebase	5	35%
07	Solutions, MSBuild & Mono-repos	Gradle Kotlin DSL · Maven multi-module · Lerna	B · Runtime & codebase	5	70%
08	ASP.NET Core Web APIs	Spring Boot · NestJS · Express	C · Services	7	75%
09	Data Access with EF Core	JPA/Hibernate · TypeORM · Flyway/Liquibase	C · Services	6	70%
10	Testing, TDD & 100% Coverage	JUnit · Mockito · MockMvc · Postman · BDD	C · Services	6	85%
11	Security & Identity with Entra ID	Spring Security · OAuth2/OIDC · Entra ID / AD B2C	D · Production	6	85%
12	Cloud-Native .NET on AWS & Modernization	AWS Lambda · ECS Fargate · Terraform · GitHub Actions · IIS estates	D · Production	8	80%
	Total			71	• Estimate

7 · Hands-on — build and run the lesson-01 samples

Three projects, two languages, one command each. L01.PlatformKit is a C# class library holding the PlatformReport record. L01.CSharpApp and L01.VbApp are console apps that reference it and print what runtime they are on. Running both is the fastest way to see that the language is a front end and the runtime is shared.

Two repository-level files make every sample build the same way. global.json pins the SDK; Directory.Build.props sets the target framework, nullable reference types for C#, Option Strict On for VB, and treats warnings as errors so no sample teaches a habit the compiler is already complaining about.

7.1 · The two files every sample inherits

JSON

global.json

```
{
  "sdk": {
    "version": "10.0.401",
    "rollForward": "latestFeature"
  }
}
```

Pins the SDK, not the runtime. latestFeature accepts 10.0.402 or 10.0.500 if 10.0.401 is missing, but never an 11.0 SDK.

```

<Project>
  <!--
    Applies to EVERY sample project in this repository: MSBuild walks up from each project folder
    and imports the first Directory.Build.props it finds. Lesson 07 explains this file line by line.
  -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Deterministic>true</Deterministic>
    <!-- bin/ and obj/ go to one place instead of beside every project -->
    <UseArtifactsOutput>true</UseArtifactsOutput>
    <ArtifactsPath>$(MSBuildThisFileDirectory).build\artifacts</ArtifactsPath>
    <!-- a sample that warns is a sample that teaches the wrong habit -->
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <!-- ...but a newly published package advisory must not break an old lesson overnight -->
    <WarningsNotAsErrors>NU1901;NU1902;NU1903;NU1904</WarningsNotAsErrors>
  </PropertyGroup>

  <PropertyGroup Condition="'$(MSBuildProjectExtension)' == '.csproj'">
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <PropertyGroup Condition="'$(MSBuildProjectExtension)' == '.vbproj'">
    <OptionStrict>On</OptionStrict>
    <OptionExplicit>On</OptionExplicit>
    <OptionInfer>On</OptionInfer>
  </PropertyGroup>
</Project>

```

Read the conditions on the last two groups. `$(MSBuildProjectExtension)` lets one file give C# projects nullable reference types and VB projects `Option Strict On`. The NU1901–NU1904 exclusion stops a newly published package advisory from failing an old lesson overnight.

7.2 · Commands

Terminal

```

# the SDK this repository pins (global.json) must be installed
dotnet --list-sdks

# from the repository root
dotnet run --project lesson-01-dotnet-platform-map/samples/L01.CSharpApp
dotnet run --project lesson-01-dotnet-platform-map/samples/L01.VbApp

# or build, test and run every sample of this lesson
python 0-script/verify_samples.py --only 1

```

7.3 · What you should see

Output — C# app

captured on the build machine

```

Hello from C# on .NET 10.0.12
os      Microsoft Windows 10.0.26200
arch    X64
JIT on  True
metadata v4.0.30319
assembly L01.CSharpApp
same?    False
copy?    True

```

Output — VB app

captured on the build machine

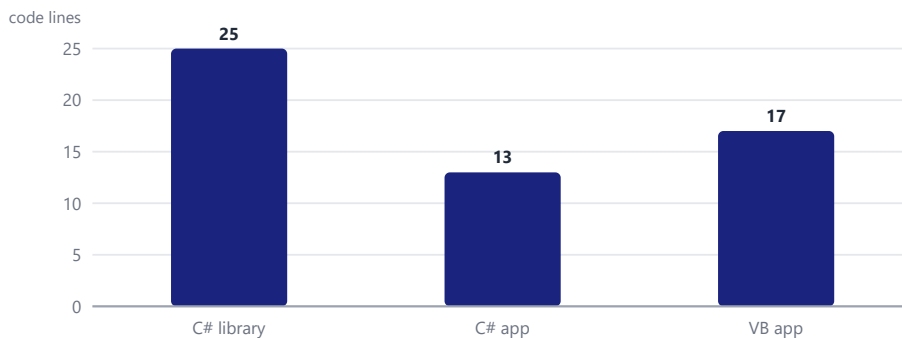
```

Hello from VB on .NET 10.0.12
os      Microsoft Windows 10.0.26200
arch    X64
JIT on  True
metadata v4.0.30319
assembly L01.VbApp
equal?   True

```

Your OS line will differ; the rest should not. `JIT on True` means a normal JIT-compiled run — publish with Native AOT and it prints `False`. `metadata v4.0.30319` appears on .NET 10 exactly as it did on .NET Framework 4.

7.4 · Lines of code in the lesson-01 samples



non-blank, non-comment lines · measured from the sample files when this PDF was built

The VB app is longer for the same behaviour — a `Module/Sub Main` wrapper, explicit line continuations and `End` keywords. Line count is a readability signal, not a quality score: VB's verbosity is part of why it stays approachable.

⚠ Three traps on day one

1. **"No .NET SDKs were found."** A runtime-only install (for example a Desktop Runtime under Program Files) earlier on PATH shadows the SDK. `dotnet --list-sdks` must list 10.0.x; if not, put the SDK folder first on PATH or set `DOTNET_ROOT`.
2. **ImageRuntimeVersion says v4.0.30319.** That is the metadata format, unchanged since .NET Framework 4. Read the target framework from the project file or `RuntimeInformation.FrameworkDescription`, never from assembly metadata.
3. **The .exe is not native code.** `L01.VbApp.exe` is an apphost that starts the runtime and loads `L01.VbApp.dll`. Copying only the .exe to another machine does not work.

✓ Checkpoint — you are ready for lesson 02 when

1. `python 0-script/verify_samples.py --only 1` reports 3/3 passed on your machine.
2. You can point at each box in diagram 2.1 and name its JVM counterpart.
3. Given a project file, you can say which row of table 3.4 it belongs to and what its support status is.
4. You can explain why the VB app can compare the C# record but cannot copy it with `with`.

🔍 Check yourself

1. Why does an assembly built for .NET 10 report `ImageRuntimeVersion v4.0.30319`, and what should you read instead to find its target framework?
2. A repository has `packages.config`, `Global.asax` and `<TargetFrameworkVersion>v4.7.2`. Which runtime does it need, and why can it not run on a Linux container unchanged?
3. On what date do .NET 8 and .NET 9 leave support, and why is upgrading from 8 to 9 not a fix?
4. Which publish mode would you choose for a latency-sensitive Lambda function, and what do you give up?
5. What can Visual Basic do with a C# record, and what can it not do?
6. Which file pins the SDK version for a repository, and which file shares build settings across its projects?

Lesson 01 in one line: .NET is a multi-language runtime shaped like the JVM — CoreCLR, IL assemblies, a tiered JIT and NuGet — shipped every November, with **.NET 10 LTS** as the target for new work and **10 Nov 2026** as the end of .NET 8 and 9. C# evolves; VB consumes.

Next: lesson 02 (C# Language Essentials) — the syntax and type-system habits that make C# read like C#.